

Beyond RISC: The PgC7600 Microprocessor

A low-cost, second-generation RISC processor
that may be fast enough to emulate other processors

DICK POUNTAIN

Ever since Sir Clive Sinclair became involved in Anamartic, the Cambridge-based wafer-scale chip firm, persistent rumors have circulated in the U.K. computer press that he is working on an ultrahigh-performance parallel processor. So far, however, those rumors have been notably short on detail. Anamartic is actually working on fast cache-memory products, while the truth behind the rumors involves another pioneer of British computer design, Chris Shelton, and a firm called PGC, in which Sinclair is a minority shareholder.

Veterans of the U.K. personal computer scene may remember Chris Shelton as the designer, back in 1977, of the Nascom single-board computer, which started a whole generation of home hackers on programming careers. Others may remember him for the Shelton SigNet business computer, launched in 1981. An advanced design for its time, the SigNet served multiple users with its close-coupled network of Z80 processors running a CP/M-compatible operating system—a sort of parallel computer.

IBM PC Emulation on a Transputer

In 1984, Sinclair and Shelton independently approached Immos to discuss using its newly born transputer as the engine for a workstation that would achieve PC compatibility via software emulation. Even though software emulation is slow, they believed that a transputer was capable of emulating a 4.77-MHz IBM XT in software with comparable performance. Sinclair and Shelton, who had been friends for years, then worked together to design a workstation that would have high-resolution monochrome graphics, not unlike those of the Mac; integrated fax, scanner, printer, and photocopier capabilities; and easy-to-use visual software that would establish it as a universal office tool.

By 1987, both men had given up on the transputer as an emulation engine. The market now demanded at least 12-MHz IBM AT performance (with the 386 coming fast), and the 20-million-instructions-per-second (MIPS) transputer was not up to the task. Also, Immos's future was looking uncertain, with a For Sale sign hanging over the firm.

Sinclair and Shelton decided instead to design a new processor to drive their office workstation. It would combine low system cost with high performance. To achieve the low component count that Sinclair's ultralow-cost approach to computer manufacturing required, this processor had to be the CPU as well as the graphics CRT controller and the laser-printer engine.



A Fresh Start

The processor, code-named the Pgc7600, has been in development since 1988 by a small team led by Shelton and financed mainly by Sinclair. Rumors persist that the firm may use the PGC chip in a successor to the Cambridge Computer 288 notebook computer. Despite Sinclair's recent sale of his interest in Cambridge Computer, Shelton insists that Sinclair is committed to PGC and that adequate funding exists to continue the project.

Computer simulations of the chip were completed by July 1989 and masks for making it by March 1990. First samples failed because of a process fault, and second samples revealed a flaw in the RAM interface. I had timed this article to coincide with the delivery of third (working) samples of the chip, but these have been delayed. Hopefully, a Pgc7600 will be running by the time this article is published.

If you view the Acorn RISC machine (Acorn ARM), MIPS, Sun SPARC, and Motorola 88000 processors as first-generation RISC chips, then the Pgc7600 architecture would represent the next generation. The key idea behind its design is that speeding up a RISC processor eventually takes you to a point at which you cannot keep up with getting signals on and off the chip. Pins simply cannot be driven fast enough to service a 200-MIPS processor, which is the sort of speed for which Shelton is aiming. He therefore has decided to isolate the RISC CPU by completely surrounding it with on-chip peripheral-control units

that handle all the communications with the outside world (e.g., memory accesses, interrupts, and I/O channels). The Pgc contains on-chip timers, video RAM (VRAM) support, interrupt-vector generation, DMA, an I2C bus for LANs, and a memory controller unit (MCU), which can refresh dynamic memories. The CPU can "talk" only to these on-chip units and knows nothing about pins and the external world.

In the resulting architecture, the CPU core is allowed to run free (i.e., unlocked) at whatever speed the silicon will support and for as long as it has instructions to process. Clocking comes from the outside world by means of the memory interface, which must, of course, be synchronous to work with present-day memory chips. Between any two external memory cycles, the processor

may execute hundreds or thousands of instructions if it is in a tight loop working on register contents only; the programmer does not need to know and, indeed, cannot know how many such instructions are executed.

As so often happens in this industry, the same idea pops up in several places at once, and sure enough, the November 1990 BYTE Microbytes article "Minimalist Architecture Promises Speed, Chips That Can Mimic Others" saw the announcement of Teraplex's Minimum Instruction Set Computer (MISC) design. Although dissimilar in many ways, it uses a similar free-running principle.

All the Pgc's interfaces to the outside world have been optimized for speed. For example, the chip uses unbuffered static-column mode for RAM accesses and a small but ingenious on-chip cache (called the Q-Cache) to keep the processor fed with instructions. In addition, to further improve the memory bandwidth for instruction fetches, the Pgc has a simple RISC core processor with about 100 instructions, almost all of which are 1 byte long.

The instructions are all hard-wired, so there is no micro-code, but the Pgc still uses an on-chip ROM area that contains subroutines composed of actual processor instructions. The hardware can cause jumps into these ROM routines, which implement CISC-type compound instructions, such as interrupt handlers, I/O data transfers, and memory-block moves. For example, flipping an interrupt pin may cause a direct jump into a ROM subroutine, providing extremely fast interrupt response. Resetting the processor is also treated as an interrupt, jumping you into ROM bootstrap code. ROM code also helps to manage the Q-Cache.

Return to Bipolar Technology

To achieve the performance (in excess of 160 MIPS) that he believes is required to emulate other commercial CPUs in software, Shelton is implementing the Pgc7600 in a bipolar process—a most unorthodox decision. Bipolar technology is as old as semiconductor technology itself; the original discrete transistors were bipolar devices made from simple sandwiches of P- and N-type silicon. The technology is familiar today in the form of Schottky TTL parts. Because of its high power consumption, however, bipolar technology has been completely supplanted by MOS technologies (NMOS and CMOS) for microprocessors and memories. Bipolar transistors consume power in the on and off states, whereas MOS transistors consume power only during the transition.

Bipolar technology does have its advantages: it is faster than MOS, and it scales down better to submicron sizes. Also, because it is current-switching rather than voltage-switching, it can drive low-impedance loads like CPU pins faster than MOS can, bringing benefits in better CPU-memory bandwidth. PGC skirts the power-consumption problem by reducing a logic transition to 0.25 volt instead of the standard TTL 5 V, which reduces the energy dissipation by a factor of 400 (since the energy depends on voltage squared).

The process that PGC uses is one originally developed by Ferranti (now Plessey-GBC), called Collector Depletion Isolation (CDI). It offers access times of 2.5 nanoseconds at the modest 1.2-micron scale. The Pgc7600 will be implemented initially as a gate array that occupies a 10- by 10-mm silicon chip, but this could be reduced to 7 by 7 mm with a custom layout. Some 33 percent of the chip area is RAM and ROM. In CMOS, Shelton believes that the 6000-gate chip could be implemented on a 2- by 2-mm die, but performance would fall to between 60 and 70 MIPS.

The bipolar process requires fewer masks than does CMOS.

continued

BYTE ACTION SUMMARY

A Faster RISC Processor

A new British company named PGC has designed what may be called a second-generation RISC processor. The Pgc7600 microprocessor will reportedly run at 160 MIPS, a speed made possible by integrating numerous support chips into the processor. The chip's designers originally intended it to be used for full-speed emulation of the Intel 8086 family of processors, but now they are looking at broader applications. In quantity, the chip will reportedly sell for US\$20. Prototype versions of the Pgc7600 should be available by the time you read this.

PGC hopes this will help them produce the chip for US\$20 in quantity. This, in combination with the fact that it requires few glue-logic chips, could make it attractive as an embedded controller as well as an almost "glueless" single-chip CPU for low-cost workstations.

A 160-MIPS CPU

The prototype Pgc7600 has an integer CPU that is expected to operate at about 160 MIPS. There is no on-chip FPU. The CPU is provided with 40 registers, divided into five banks of eight.

The bipolar process could make it attractive as an embedded controller as well as an almost "glueless" single-chip CPU for low-cost workstations.

All are 32 bits wide and can be accessed in 2.5 ns, giving a processor cycle time of about 5 ns. There is also an accumulator, a program counter, and a few special registers, including a status record.

The register architecture is designed to be scalable in future versions while maintaining compatibility with earlier versions. To achieve this, the register banks are named 0, 1, 2, 3, and TOP. When an interrupt is serviced, state information such as the Accumulator and Program-Counter contents are saved only in the TOP register bank. Designers could add more banks (numbered, for example, 0, 1, 2, 3, 4, 5, and TOP) without causing programs written for earlier versions to break.

The Q-Cache is a fast (2.5-ns) on-chip RAM area that is 32 bytes long and can, therefore, hold an average of 32 instructions. In hardware terms, the Q-Cache is implemented as a circular buffer. During sequential program execution, it gets reloaded in single-word (32-bit) chunks, thereby acting as a 32-byte moving window into main memory. When a branch instruction causes the cache to become invalid, it gets reloaded in 64-bit chunks, and each chunk can start executing while the next one is loading.

This is a simple caching scheme, without set associativity or the other elaborate features used by manufacturers such as Intel. The Q-Cache is transparent to programs, which see only a normal program counter; there are no cache-flush instructions, and none are needed. Like the register architecture, the Q-Cache would be transparently scalable for future chip versions. Programs written for some larger cached version would always run on old versions but perhaps less efficiently, as some loops would not fit in the cache.

The instruction fetch unit can supply instructions to the CPU either from the Q-Cache or from the on-chip ROM, and the CPU rather than the IFU decides when it is ready for the next instruction. All CPU instructions transfer data between registers, the accumulator, and the integer ALU, and their timing

values are purely nominal; in practice, the speed at which they execute varies with temperature and from device to device according to the silicon quality. If the CPU runs out of instructions, it just waits. The IFU executes program control instructions (e.g., branches) and also obtains further instructions when an interrupt occurs or when the Q-Cache becomes empty or invalid; it does this by talking to another peripheral unit, for example, the MCU.

The MCU is described in PGC's documentation as aggressive, and this is no exaggeration. It controls most of the Pgc7600's 84 external pins, and it features separate address and data buses. Since the Pgc7600 was designed for use in low-cost systems, it clearly could not depend on the use of fast and expensive static RAM to attain the necessary processor-to-memory bandwidth. Hence, the MCU uses every possible technique to optimize access to ordinary DRAM, and it supports the static-column and fast-page modes of modern DRAM chips as well as SRAM. These modes (also supported by the Acorn ARM and Intel i860) typically allow 512 consecutive data words to be accessed from the same row of a RAM chip by changing the column address but not the row address. For inexpensive, 100-ns-access-time DRAM chips, the row-address select cycle time may be as much as 180 ns, but the column-access select cycle is only 55 ns, becoming the effective cycle time. The Pgc7600 generates its CAS/RAS multiplexer signals on-chip, so they add only 3 ns to the cycle time. With an external MMU chip, as much as 20 ns might be consumed in communication between the CPU and the MMU. The use of a bipolar drive for the pins allows the MCU to access memory without buffering, saving even more time.

The prototype Pgc7600 will access memory organized into two banks (later versions may use four or more), addressable on random 32-bit word boundaries; byte addressing is not supported. The MCU has direct access to the accumulator; thus, it always knows what the next address will be, making for fast bank logic. It can even handle two banks of different speed RAM (e.g., one fast SRAM and one slow DRAM). The timing information gets loaded from an external ROM during the bootstrap interrupt procedure. The MCU is the only part of the Pgc7600 that depends on external timing. With a 16-MHz clock, it should achieve a 25-ns cycle time with 15-ns SRAM and a 50-ns cycle time with 80-ns DRAM, providing a bandwidth between 160 and 80 megabytes per second.

The MCU doesn't directly support memory protection or virtual memory, but it provides indirect support by being interruptible, even in midcycle. Hence, external detection hardware can jarr the controller when a page fault or protected access occurs. The CPU then runs out of instructions and waits patiently while an interrupt routine corrects the fault, for example, by requesting a new page from a disk controller. Shelton suggests that a second Pgc7600, mounted pin-for-pin over the master processor, is the ideal engine to act as a virtual memory controller, but even a Z80 could do the job.

Although the original requirement for the Pgc7600 to act as a full graphics processor was abandoned, the final design can still act as an integrated CRT controller. A dedicated interrupt causes a jump into a ROM subroutine, which grabs a video address from the TOP register bank, puts it into the MCU, and then requests a VRAM access cycle. Control then returns to external code, which computes the next video address. If you put your video-synchronous signal onto this interrupt pin, the Pgc7600 becomes a CRT controller for video buffers implemented with VRAM chips (but not ordinary RAM). Shelton estimates that controlling a 1024- by 768-pixel by 8-bit video buffer in this way would consume about 1 percent of CPU time.

continued

Beyond Emulation

The original purpose of the high performance promised by the PpC7600 was for full-speed emulation of other processors, especially the Intel 8086 family, in software, allowing the chip to drive a low-cost IBM PC-compatible workstation. However, PpC is now envisaging broader applications. Though the chip does not have special communications hardware like that of the Taurus transputer, its fast memory access would enable it to be used in parallel-processing systems where shared memory is the communication channel between processes.

Alternatively, you could build a message-passing parallel system out of distributed processors with dual-ported local memories. Your software would assemble a message in RAM and then raise a PpC message interrupt, which uses the same mechanism as the video interrupt mentioned above. Using a simple external clock, the dual-port message RAM then transfers its contents serially into another remote RAM, just as a VRAM would transfer its contents to a CRT display. The software does not need to know anything about the physical carrier layer, which might be a ribbon cable, a bus, or a fiber-optic link. Apart from the microsecond it takes to set up the transfer, the CPU is not involved, and it can continue processing while the message is autonomously transferred.

Unlike transputer links, this scheme places no constraints on the topology; you can have as many message channels as you need, and they can be broadcast or point-to-point. This sort of system would be ideal for running distributed message-passing operating systems such as Helios or Tacos (see my article "Tacos: An Innovation in Operating Systems" on page 117). PpC has

been in close contact with the developers of both operating systems and has even modified the instruction set of the PpC to better accommodate Tacos's message-passing scheme.

As mentioned, the PpC7600 has no hardware for floating-point arithmetic, which would seem to disqualify it from supercomputer applications. However, Shelton argues with some conviction that the superior processor-to-memory bandwidth can be exploited here. The basic problem in high-speed floating-point computing is always data bandwidth, suggesting as a solution that the PpC7600 feed floating-point data into pseudoregisters held in dual-ported RAM. From there, the data would be autonomously transferred into a streamed FPU, such as those from AMD, Weitek, or Cyrix, and the answers would be transferred back to the PpC by the same means. Using 15 ns SRAM pseudoregisters, the PpC7600 should be capable of transferring data at 160 Mbits or 1280 megabits per second. If each floating-point operation involved 100 bits, a three-operand scheme could sustain the quoted peak rate of the FPU, which could be up to 33 million single-precision floating-point operations per second. Combining this with the parallel scheme would allow pipelines of many PpC7600s to be constructed to deliver supercomputer performance.

Another intriguing possibility, given Sinclair's Anamorphic connections, would be wafer-scale integration, for which the 6000-gate PpC design is an ideal candidate. We'll see what happens. ■

Dick Pountain is a contributing editor for BYTE based in London. He can be reached on BIX as "dickp."

More Europeans Read BYTE Than Any Other Pan-European Computer Magazine!

Europe's most advanced computing experts, over 95,000, have always relied on BYTE's authoritative editorial for making critical purchasing decisions. Only BYTE provides these top-level decision makers with the objective information they need on the latest products and technologies in both Europe and the United States. More than 1 million people worldwide read BYTE every month.

BYTE

**Visit BYTE at CeBIT, March 13-20, Hall 6 Stand H30
and Receive Your Free Copy of the March Issue!**

